Zivra Vision

Multimodal AI Text-to-Image Generation Studio

AMD AI DevMaster Hackathon 2026
Track: Multimodal AI

Team: Zivra
Submitted by: Sangamithra

Project Report

1. Abstract

Zivra Vision is a multimodal AI application that converts natural-language text prompts into original images using the Stable Diffusion v1.5 latent diffusion model. The system is built with a Gradio web interface for accessibility and is fully compatible with AMD Radeon GPUs through the ROCm software stack, in addition to standard CUDA and CPU execution. The project was developed and demonstrated for the AMD AI DevMaster Hackathon to showcase practical, GPU-accelerated generative AI that runs unmodified across heterogeneous hardware.

2. Introduction

Text-to-image generation is one of the most visible applications of modern generative AI, allowing users to describe a scene in plain language and receive a corresponding visual output. Zivra Vision packages this capability into a simple, self-contained web application that anyone can run locally or in the cloud, with no external API dependency: the diffusion model runs entirely on the user's own hardware.

3. Problem Statement

Most accessible text-to-image tools rely on hosted APIs that involve cost, rate limits, and data-privacy trade-offs, while self-hosted diffusion pipelines are often hardware-locked to NVIDIA CUDA, leaving AMD GPU owners without an easy on-ramp. There is a need for a lightweight, open, self-hosted image generation tool that runs equally well on CPU, NVIDIA CUDA, and AMD ROCm hardware without code changes.

4. Objectives

- Build a working text-to-image generator using an open-source diffusion model.
- Provide an accessible, no-code web interface for prompt entry and image viewing.
- Ensure the application runs on CPU as a fallback with no code changes.
- Design the code to be Radeon/ROCm-compatible so it accelerates automatically on AMD GPUs.
- Keep the codebase minimal and readable for hackathon review and reproducibility.

5. Proposed Solution

Zivra Vision loads the pretrained Stable Diffusion v1.5 model via Hugging Face's **diffusers** library and exposes it through a Gradio Blocks interface. The application detects available hardware at runtime: if a CUDA-capable or ROCm-enabled GPU is present, the model loads in half precision (float16) with attention slicing enabled for lower memory use; otherwise it falls back to full-precision CPU inference automatically. Users control the prompt, number of inference steps, and guidance scale directly from the browser, and the generated image is returned inline.

6. System Architecture

The application follows a simple three-layer flow: a Gradio front end collects user input, a Python inference layer drives the Stable Diffusion pipeline, and the underlying hardware layer (CPU, CUDA, or ROCm) executes the diffusion computation.

Layer	Component	Responsibility
Interface	Gradio Blocks UI	Collects prompt, steps, guidance scale; displays output image
Application	generate() function	Invokes the diffusion pipeline with user parameters
Model	StableDiffusionPipeline (diffusers)	Runs the denoising loop to synthesize the image
Hardware	CPU / CUDA / ROCm	Executes tensor operations; GPU path auto-selected

7. Technology Stack

- **Language:** Python 3
- **Model:** Stable Diffusion v1.5 (runwayml/stable-diffusion-v1-5)
- **Core library:** Hugging Face diffusers, PyTorch
- **Interface:** Gradio Blocks
- **Acceleration:** AMD ROCm-enabled PyTorch / NVIDIA CUDA / CPU fallback

8. AMD Radeon GPU & ROCm Integration

Zivra Vision does not hard-code any NVIDIA-specific logic. Device selection is done through PyTorch's standard **torch.cuda.is_available()** check, which returns True on both CUDA and ROCm-enabled PyTorch builds, since ROCm exposes the same CUDA-compatible interface. To run on an AMD Radeon GPU, the only change required is installing the ROCm build of PyTorch:

```
pip install torch torchvision torchaudio --index-url
https://download.pytorch.org/whl/rocm6.2
```

Once installed, the application automatically loads the model in float16 precision onto the Radeon GPU and enables attention slicing to reduce VRAM usage, with no changes to app.py.

9. Workflow

- User enters a text prompt and adjusts inference steps and guidance scale in the browser.
- The Gradio app calls the generate() function with these parameters.
- The StableDiffusionPipeline runs the denoising diffusion loop on the detected device.
- The resulting image tensor is decoded and returned to the browser for display.

10. Features

- Text-to-image generation from free-form natural language prompts.
- Adjustable inference steps (10–50) and guidance scale (1–10) for quality/creativity control.
- Automatic GPU acceleration (CUDA/ROCm) with seamless CPU fallback.
- Attention slicing for reduced GPU memory footprint.

- Simple, shareable Gradio web interface (public link support via share=True).

11. Results

The application was developed and demonstrated on CPU hardware due to hackathon time constraints on GPU provisioning. The pipeline, code path, and dependency stack are fully ROCm-compatible and require no code modification to run on an AMD Radeon GPU — only the PyTorch installation target changes. On CPU, the pipeline successfully generates coherent images from text prompts, validating the correctness of the model integration and interface; GPU execution is expected to reduce generation time from the CPU baseline of roughly a minute per image to a few seconds per image, consistent with typical Stable Diffusion GPU speedups.

12. Future Scope

- Benchmark and document actual inference latency on AMD Radeon GPUs via ROCm.
- Add support for newer/faster models such as SDXL-Turbo for near-real-time generation.
- Add negative prompts, seed control, and batch generation for more creative control.
- Add image-to-image and inpainting modes to extend multimodal capability.
- Containerize the application (Docker) for one-command ROCm deployment.

13. Conclusion

Zivra Vision demonstrates a compact, hardware-portable approach to generative text-to-image AI. By relying on standard PyTorch device APIs rather than vendor-specific code, the project runs unmodified across CPU, NVIDIA, and AMD Radeon GPU environments, directly addressing the hackathon's goal of practical AMD GPU-accelerated AI applications.

14. References

- Rombach, R. et al. High-Resolution Image Synthesis with Latent Diffusion Models (Stable Diffusion).
- Hugging Face Diffusers Documentation: https://huggingface.co/docs/diffusers
- AMD ROCm Documentation: https://rocm.docs.amd.com
- Gradio Documentation: https://www.gradio.app/docs
- PyTorch ROCm Installation: https://pytorch.org/get-started/locally/

15. Appendix: requirements.txt

```
torch
diffusers
transformers
accelerate
gradio
safetensors
```